PROJECT-BASED LEARNING

PHASE-II EVALUATION

IntelliPlag

Team ID: DSCPP-III-2026-T069

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T069
Semester: 3rd
Section: DS1, DS2, ML1
Domain: CSE

Team Members

1. Shivya Sinha – 2510380257
2. Srishti Kuletha – 2510012545
3. Antriksh Singh Jagwan – 2512510006

Mentor

Ms. Shrishti Chamoli

Interactions completed: 7

Phase-I Recap & Feedback Incorporated

Phase-I Status

IntelliPlag was proposed as a plagiarism detection engine designed to tokenize documents, generate N-Grams, and apply DSA-based string processing algorithms to calculate a overall plagiarism percentage.

Feedback Received

- Develop project module-by-module.
- Test every component independently.
- Deal with handling a database to check a document against.

Changes Incorporated

- Structured code into isolated component files.
- Added individual test routines for DSA concepts.
- Made an SQLite database to handle dataset.

Problem Statement & Finalized Objectives

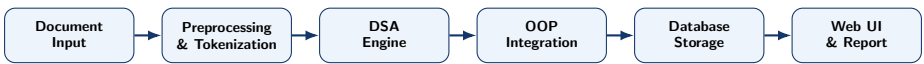
Problem Statement

Manual detection of text similarity across academic/digital files is slow and inefficient. IntelliPlag automates this by comparing multi-document texts through custom DSA engines and visualizing similarity reports.

Finalized Objectives

1. Preprocess and tokenize input text documents efficiently.
2. Utilize Tries, Hash Tables, and Vectors for memory representation.
3. Generate N-Grams and perform string/phrase matching.
4. Calculate overall similarity percentage and highlight exact matches.
5. Generate structured plagiarism analysis reports.

Updated System Architecture / Workflow



Major Components

- Text preprocessing & tokenization
- Trie, Hash Table, and Vector engines
- N-Gram calculation string matching
- Similarity scoring algorithm

Changes Since Phase-I

- Made individual DSA files for clear working.
- More OOP's concepts applied.

Module Design & Functional Requirements

Major Modules

1. Text Preprocessing Module
2. DSA-Based Detection Engine
3. OOP Integration Layer
4. Web UI & Reporting
5. Database Management Layer

Functional Requirements

- Multi-document input uploading
- Text normalization and token separation
- Hash/Trie-based pattern matching
- Similarity percentage computation
- Interactive report rendering

Module Dependency

Web UI passes document content to backend processing layer → DSA modules compute token similarities → OOP layer structures report objects → DB stores execution metadata.

Technology Stack & Development Methodology

Technology Stack

- **DSA Engine:** C
- **OOP Architecture:** C++
- **Data Analytics:** Python
- **Frontend:** HTML5, CSS3, JavaScript
- **Database:** MySQL
- **Tooling:** VS Code, Git, GitHub

Development Methodology

1. System Analysis & Design
2. C DSA Module Implementation
3. C++ OOP Class Architecture
4. Web UI API Development
5. Database Integration
6. Comprehensive System Testing

Implementation of PBL Course Concepts

Course	Concepts Implemented	Target Module
Data Structures in C	Arrays, Dynamic Vectors, Hash Tables, Tries, String Matching, N-Grams	DSA Detection Engine
OOPs with C++	Classes, Encapsulation, Inheritance, Polymorphism, Modular Components	Document processing & Engine wrappers
Machine Learning	Feature extraction, Tokenization, Similarity Metrics	Text Analysis
DBMS	Schema Design, Query Execution, Document	Database Layer

Core Integration: Data Structures execute core string match logic, wrapped inside OOP C++ objects.

Development Progress

Module	Status	Details / Milestones
Text Preprocessing	In Progress	Case-folding, punctuation strip, and token stream generation completed.
DSA Engine	In Progress	Trie construction and N-Gram hashing algorithm implementation under optimization.
OOP Integration	In Progress	C++ classes designed for document wrapper and similarity score calculation.
Web UI	In Progress	File selection page, percentage display screens under active layout construction.
Database	Planned	MySQL schema designed for storing execution results and logs.

Overall Project Completion: 80%

Working Prototype / Demonstration

Implemented Features

- Document parsing and intake
- Text normalization pipeline
- Token generation N-Gram setup
- Similarity scoring algorithm
- Section match highlighting

Prototype/Output

[Insert Web UI Screenshot / Terminal Output]

Expected Demonstration:
Upload Screen / Similarity Percentage Output

Testing, Results & Validation

Test/Module	Expected Result	Observed Result	Status
Preprocessing	Clean token sequence free of special characters	Cleaned string arrays	Passed
Tokenization	Words mapped into indexing arrays	Correct token array indexing	Passed
N-Gram Engine	Exact phrase overlap identified via hashing	Pending evaluation run	Pending
Similarity Metric	Similarity percentage accurately reported	Pending evaluation run	Pending
Web UI Dashboard	Interactive visual presentation of match stats	Rendered layout intact	In Progress

Result/Validation

System is validated on algorithm response time, similarity accuracy percentages, and match identification precision.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: https://github.com/shivyasinha06-sudo/DSA-OOPs_pbl.git
- README.md: Updated
- Contributors: All Members Added
- Repository Status: Active

Development Activity

- DSA C engine commits
- OOP C++ interface commits
- UI layout updates
- Integration testing scripts

Repository verified and active with regular team commits.

Individual Contributions

Member	Role	Technical Contribution	GitHub Commits
Shivya	Core DSA Engine	C Data Structures, Tries, Hash Tables, N-Gram calculations, and String Matching modules.	Tracked
Srishti	OOP Architect	C++ Encapsulation wrappers, modular class structure, and C-to-C++ engine bridge.	Tracked
Antriksh	Web UI & DB	Web UI layout (HTML/CSS/JS), backend endpoint handling, and MySQL database setup.	Tracked

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Large Input Overhead: Addressed using Trie string searching and hash lookup.
- C / C++ Interoperability: Managed via clean extern interface wrapper designs.
- Web Integration: Using dynamic file parsing to stream data cleanly.

Solutions / Learning

- Indexed large dataset to a few thousands.
- Developed clear header interfaces.

Phase-III Roadmap

1. Finalize string detection speed optimizations.
2. Complete full UI to C++ backend pipeline.
3. Integrate MySQL execution history database.
4. Perform stress testing on heavy documents.
5. Finalize plagiarism report export functions.

Expected Outcomes & References

Expected Outcomes

- High-speed multi-document plagiarism analysis tool.
- Clear percentage scoring and highlighted exact phrase overlaps.
- Functional demonstration of DSA concepts combined with clean C++ OOP architecture.

Key References

1. T. H. Cormen et al., *Introduction to Algorithms*, MIT Press.
2. B. Stroustrup, *The C++ Programming Language*, Addison-Wesley.
3. M. T. Goodrich et al., *Data Structures and Algorithms in C++*, Wiley.

Thank You!

Questions & Discussion